Модуль 6. Реверс-инжиниринг и бинарные уязвимости

Введение

Модуль посвящён введению в реверс-инжиниринг и базовым понятиям бинарных уязвимостей. В ходе изучения материала вы познакомитесь с основными подходами к анализу программ, научитесь различать статическое и динамическое исследование, разберётесь в устройстве исполняемых файлов и получите представление о том, как ошибки в низкоуровневом коде могут приводить к критическим уязвимостям.

Особое внимание в модуле уделяется связи между устройством программы и её поведением во время выполнения. Мы последовательно пройдём путь от общего понимания реверс-инжиниринга до знакомства с архитектурой x86_64, устройством стека и принципами возникновения переполнения стека. Такой подход позволяет не только освоить основные инструменты анализа, но и заложить основу для дальнейшего изучения эксплуатации бинарных уязвимостей.

Как полученные знания пригодятся в профессиональном развитии

$term

Реверс-инжиниринг — это исследование устройства программы или системы, часто без доступа к исходному коду, с целью восстановить логику работы, понять внутреннее устройство и при необходимости изменить поведение программы.

$end

$term

Crackme — это учебная или соревновательная программа, в которой исследователю необходимо понять механизм проверки пароля и подобрать корректный ввод, который будет принят программой.

$end

Навыки, полученные в рамках этого модуля, востребованы в анализе вредоносного программного обеспечения, пентестинге, аудите безопасности, исследовании чужих приложений и решении задач CTF. Кроме того, они формируют фундамент для более глубокого изучения системного программирования, защиты программ и практик безопасной разработки.

Цели и результат модуля

Студент, освоивший программу модуля, должен:

1. Понимать назначение и задачи реверс-инжиниринга.
2. Различать статический и динамический анализ программ.
3. Понимать связь между машинным кодом, ассемблером и языками высокого уровня.
4. Анализировать исходный код, байт-код и исполняемые файлы.
5. Использовать базовые инструменты реверса и отладки: IDA, Ghidra, GDB, pwndbg, ltrace, strace, pwntools.
6. Понимать основы архитектуры x86_64.
7. Разбираться в назначении регистров и принципах передачи аргументов в функции.
8. Объяснять устройство стека и стекового кадра.
9. Понимать природу типичных бинарных уязвимостей.
10. Объяснять механизм возникновения переполнения стека и роль защитных механизмов.

Урок 1. Введение в реверс-инжиниринг

Первый урок посвящён формированию общего представления о реверс-инжиниринге как о дисциплине. На этом этапе важно не просто дать определение термину, а показать, зачем вообще нужен анализ программ без исходного кода, в каких областях он применяется и какие формы может принимать исследуемый объект.

Реверс-инжиниринг применяется в тех случаях, когда необходимо восстановить логику работы программы, понять алгоритмы обработки данных, проанализировать потенциально вредоносное поведение или выявить особенности реализации. Особенно часто такие задачи встречаются в исследовании вредоносного ПО, пентестинге, прикладном аудите безопасности, анализе чужих бинарных файлов и в задачах категории Reverse на соревнованиях CTF.

$term

Статический анализ — это исследование программы без её запуска, по исходному коду, байт-коду, машинному коду или структуре исполняемого файла.

$end

$term

Динамический анализ — это исследование программы во время выполнения, при котором анализируется её фактическое поведение, обращения к памяти, вызовы функций и системные взаимодействия.

$end

Статический анализ позволяет увидеть структуру программы в целом, оценить взаимосвязи между функциями, изучить алгоритмы и данные без влияния среды исполнения. Динамический анализ, напротив, показывает поведение программы в реальных условиях: какие инструкции выполняются, какие значения попадают в память и как изменяется состояние процесса во времени.

На практике эти подходы не противопоставляются друг другу, а используются совместно. Статический анализ помогает сформировать гипотезу о поведении программы, а динамический — подтвердить или опровергнуть её на реальном выполнении.

Реверс-инжиниринг находит применение в различных областях информационной безопасности:

- **Исследование вредоносного ПО** — анализ поведения вирусов, троянов и других вредоносных программ для понимания механизмов заражения и создания средств защиты.
- **Пентестинг** — поиск уязвимостей в программном обеспечении путём анализа его внутреннего устройства.
- **Анализ продуктов конкурентов** — изучение реализации функциональности в коммерческих продуктах для понимания использованных подходов.
- **Соревнования CTF** — решение задач категории Reverse, где необходимо восстановить алгоритм проверки пароля или найти скрытый флаг.

$imp

Важно помнить о юридических аспектах реверс-инжиниринга. В России и многих других странах существуют законодательные ограничения на обратную разработку коммерческого программного обеспечения. Всегда проверяйте лицензионное соглашение и применимое законодательство перед началом анализа. Об этом можно почитать в статье [Право на реверс. Как обратная разработка выглядит с юридической точки зрения — Хакер]

$end

Чтобы понимать реверс-инжиниринг, необходимо разобраться, в каком виде программа существует для процессора.

$term

Машинный код — это набор инструкций в числовом виде, который непосредственно исполняется процессором.

$end

Машинный код неудобен для восприятия человеком, поскольку каждая команда представлена последовательностью байтов. Для того чтобы сделать работу с низкоуровневым кодом более понятной, используются мнемонические обозначения инструкций. Так возникает язык ассемблера.

$case

Например, операция сложения в машинном коде представляется набором байтов (номер операции + операнды), а в ассемблере записывается в более понятной форме:

add 2, 2

Основные мнемоники x86:

- Сложение (№1) — add
- Вычитание (№2) — sub
- Сравнение (№3) — cmp
- Перемещение в памяти (№4) — mov

Такое представление делает возможным ручной анализ программы и служит основой для работы большинства инструментов реверса.

$end

$term

Ассемблер — это программа, переводящая ассемблерный код в машинный код.

$end

$term

Дизассемблер — это программа, преобразующая машинный код в ассемблерное представление, удобное для анализа человеком.

$end

Основные ассемблеры на Linux: **nasm**, **yasm**, **fasm**, **gas**. Существуют два основных синтаксиса: Intel и AT&T. Наиболее удобные дизассемблеры: IDA, Ghidra, Radare2.

С ростом числа архитектур и усложнением программирования появилась необходимость использовать более удобные языки программирования высокого уровня, которые скрывают низкоуровневые детали. Языки бывают компилируемыми и интерпретируемыми.

$term

Компиляция — это процесс перевода программы с языка высокого уровня в более низкоуровневое представление с помощью перезаписи команд высокого уровня эквивалентными блоками команд низкого уровня.

$end

$term

Интерпретация — это выполнение программы специальной средой, которая читает и исполняет код без его предварительного полного преобразования в машинный код.

$end

$term

Декомпилятор — это инструмент, который пытается восстановить высокоуровневое представление программы из байт-кода или машинного кода.

$end

Понимание этой цепочки важно для реверс-инжиниринга, поскольку анализируемая программа может быть представлена в разных формах: исходный код, байт-код виртуальной машины или готовый скомпилированный бинарный файл.

Таким образом, объектами реверса могут быть:

- исходные тексты программ;
- байт-код;
- исполняемые файлы;

Если у исследователя есть исходный код, задача упрощается: можно читать программу напрямую, запускать её части, модифицировать поведение и проверять гипотезы. Если доступны только скомпилированные файлы, в работу вступают дизассемблеры, декомпиляторы и отладчики.

Реверс байт-кода

$term

Байт-код — это промежуточное представление программы, предназначенное для выполнения виртуальной машиной, а не процессором напрямую.

$end

Байт-код обычно проще анализировать, чем машинный код, поскольку в нём часто сохраняются структуры классов, методов и другие элементы исходной программы. Это характерно для Java, C# и частично Python.

Характеристика	**Байт-код**	**Машинный код**
Исполнение	Виртуальная машина	Процессор
Декомпиляция	Почти полное восстановление	Частичное восстановление
Сохранение структуры	Классы, функции, имена	Имена могут быть потеряны
Сложность анализа	Проще	Сложнее

Table 1: Сравнение байт-кода и машинного кода

Инструменты для получения байт-кода:

- **Java:** jadx — для `.class`, `.jar`, `.apk` файлов
- **C#:** dnSpy — для `.exe`, `.dll` файлов
- **Python:** decompile3 (старые версии), uncompyle6 (примерно до версии 3.7), pycdc (примерно до версии 3.10), pylingual (подходит для современных версия) — для `.pyc` файлов

Реверс исполняемых файлов

Исполняемые файлы сложнее для анализа, поскольку они уже адаптированы под конкретную архитектуру и операционную систему. Для их исследования необходимо понимать формат бинаря, устройство памяти процесса, этапы компиляции и линковки, а также особенности архитектуры процессора.

Основные архитектуры процессоров:

- **X86 и X86_64** — Intel, AMD — архитектура CISC (Complex Instruction Set Computing)
- **ARM** — Apple, Samsung, Qualcomm, Nvidia — архитектура RISC (Reduced Instruction Set Computing)
- **MIPS** — RISC-архитектура

Форматы исполняемых файлов:

- **Windows** — PE (Portable Executable): `.exe`, `.dll`, `.sys`
- **Linux** — ELF (Executable Linking Format)
- **macOS** — Mach-O

$term

ELF — это формат исполняемых файлов в Linux, который представляет собой контейнер с кодом, данными и информацией о загрузке программы в память.

$end

Основные секции ELF:

- `.text` — машинный код программы
- `.rodata` — константы (read-only data)
- `.data` — инициализированные глобальные переменные
- `.bss` — неинициализированные глобальные переменные

Понимание процесса компиляции критически важно для эффективного реверс-инжиниринга. Зная, как исходный код превращается в исполняемый файл, можно лучше интерпретировать результаты дизассемблирования.

Компиляция программы на языке C состоит из четырёх основных этапов:

1. **Препроцессинг** — раскрытие директив `\#include`, макросов `\#define`, удаление комментариев
2. **Компиляция** — трансляция исходного кода в ассемблерный код
3. **Ассемблирование** — трансляция ассемблерного кода в машинный, создание объектного файла
4. **Линковка** — связывание объектных модулей и добавление стандартных библиотек

$term

Статическая линковка — всё включается в один исполняемый файл. Преимущества: проще распространять. Недостатки: больший размер, возможные проблемы с лицензиями.

$end

$term

Динамическая линковка — исполняемый файл использует функции из библиотеки, которая находится в отдельном файле. При запуске загрузчик читает таблицу импорта и отображает необходимые сегменты в память.

$end

Инструменты для реверс-инжиниринга

Для анализа бинарных файлов применяются специализированные инструменты. Наиболее известными являются IDA Pro и Ghidra. Они позволяют открывать исполняемые файлы, строить графы управления, изучать функции, просматривать строки, дизассемблированный код и псевдокод, а также постепенно восстанавливать смысл программы.

IDA Pro

Преимущества:

- Высокая стабильность работы
- Удобный интерфейс
- Поддержка множества архитектур и форматов
- Большое количество плагинов

Недостатки:

- Высокая стоимость (есть бесплатная версия с ограничениями)

Основные горячие клавиши IDA Pro:

- N — переименовать переменную или функцию
- Ctrl + F — различные виды поиска
- Y — изменить тип функции или переменной
- **Двойной клик** — перейти к коду функции или расположению переменной
- F5 — открыть псевдокод
- Пробел — переключиться между графом и линейным представлением
- X — посмотреть, где используется переменная или функция
- Наличие IDAPython для автоматизации анализа

Ghidra

Преимущества:

- Полностью бесплатна и с открытым исходным кодом

- Удобная разработка собственных плагинов
- Поддержка множества архитектур

Недостатки:

- Менее удобный интерфейс по сравнению с IDA Pro
- Иногда менее стабильна

$imp

Важно помнить, что реверс-инжиниринг — это не механическое нажатие кнопок в декомпиляторе, а аналитическая работа. Инструмент помогает увидеть код, но выводы о логике программы всегда делает исследователь.

$end

Занятие 2. Динамический анализ и бинарные уязвимости

Во втором занятии акцент переносится с устройства программы на наблюдение за её поведением во время выполнения. Динамический анализ особенно полезен в тех случаях, когда статической картины недостаточно.

Кроме того, именно динамический анализ тесно связан с практикой исследования бинарных уязвимостей. Чтобы понять, как возникает ошибка памяти или как изменяется состояние процесса под воздействием некорректного ввода, необходимо видеть реальные значения в регистрах, стеке, куче и аргументах функций.

Инструменты динамического анализа

Для базового динамического анализа применяются различные инструменты.

$term

ltrace — это инструмент, позволяющий перехватывать вызовы библиотечных функций, выполняемых программой.

$end

С его помощью можно понять, какие функции стандартных библиотек вызывает приложение, какие строки передаются в printf, какие данные участвуют в сравнении и как программа использует библиотечную инфраструктуру.

Примеры использования ltrace:

```
ltrace ./main
ltrace -e function_name ./main # отслеживать конкретные
```

```
функции
ltrace -f ./main # трассировка дочерних процессов
```

$term

strace — это инструмент, позволяющий отслеживать системные вызовы программы при взаимодействии с операционной системой.

$end

Он особенно полезен для понимания того, какие файлы открывает программа, какие данные читает и записывает, как создаёт процессы, отображает память и обращается к ядру.

Примеры использования strace:

```
strace ./main
strace -e trace=open,read,write ./main # отслеживать
отдельные системные вызовы
```

GDB и pwndbg

$term

GDB — это консольный отладчик, который позволяет управлять выполнением программы, ставить точки останова, выполнять код по шагам и исследовать память, регистры и переменные.

$end

$term

pwndbg — это плагин для GDB, который делает отладку более удобной и наглядной, особенно в задачах по бинарной эксплуатации.

$end

С помощью отладчика можно не только изучать ход выполнения программы, но и активно вмешиваться в него: менять значения переменных, наблюдать за участками памяти, отслеживать обращения к данным и анализировать стек вызовов.

Основные команды GDB:

- `gdb ./main` — запуск отладчика
- `break main` — установка точки останова на функции main
- `run` — запуск программы
- `next` — выполнение следующей строки (без захода в функции)

- step — выполнение следующей строки (с заходом в функции)
- continue — продолжение выполнения до следующей точки останова
- print argc — вывод значения переменной
- x/\<n\>\<f\>\<u\> \<адрес\> — просмотр памяти
- watch x — установка watchpoint на переменную

Автоматизация с помощью pwntools

При анализе бинарных уязвимостей часто требуется многократно запускать программу, отправлять в неё данные, считывать ответы, формировать полезную нагрузку и работать с адресами.

$term

pwntools — это библиотека Python для автоматизации взаимодействия с бинарными файлами и удалёнными сервисами в задачах по эксплуатации уязвимостей.

$end

Она позволяет запускать локальные процессы, подключаться к удалённым сервисам, отправлять и принимать данные, упаковывать адреса в нужном формате и работать с информацией о структуре ELF-файлов.

Примеры использования pwntools:

```
from pwn import *
```

Запуск локального процесса

```
p = process('./main')
```

Подключение к удалённому хосту

```
p = remote('example.com', 1337)
```

Отправка данных

```
p.sendline(b'Hello')
```

Получение данных

```
data = p.recv()
```

Упаковка адреса

```
payload = p64(0x401000)
```

Работа с ELF

```
elf = ELF('./main')
print(hex(elf.symbols['main']))
```

Введение в бинарные уязвимости

$term

PWN — это направление, связанное с эксплуатацией уязвимостей в скомпилированных программах, обычно с целью получить управление выполнением, выполнить произвольный код или получить доступ к системе.

$end

$term

Бинарная уязвимость — это ошибка в скомпилированной программе, возникающая из-за некорректной работы с памятью, адресами, типами данных или арифметикой и способная повлиять на ход выполнения программы.

$end

Бинарные уязвимости особенно характерны для языков, где программист работает с памятью напрямую. В первую очередь речь идёт о языке C, который предоставляет полный контроль над указателями, буферами и адресами, но не гарантирует автоматическую защиту от ошибок.

Такие ошибки возникают, когда программа:

- не проверяет длину входных данных;
- некорректно работает со строками;
- небезопасно форматирует вывод;
- ошибается в арифметике размеров или индексов;
- нарушает границы выделенной памяти.

В результате ошибка превращается в повреждение памяти.

$term

Memory corruption — это некорректное изменение содержимого памяти программы, вызванное ошибкой при записи, чтении или интерпретации данных.

$end

Адресное пространство процесса

Для понимания бинарных уязвимостей нужно представлять, как устроена память процесса. Разные участки программы располагаются в разных областях адресного пространства:

- код программы хранится в секции `.text`;
- глобальные данные находятся в `.data` и `.bss`;
- локальные переменные и адреса возврата — в стеке;
- динамически выделенная память — в куче;
- подключённые библиотеки загружаются в отдельные области памяти.

Такое разделение имеет принципиальное значение, потому что разные классы уязвимостей затрагивают разные участки памяти и по-разному влияют на поведение программы.

Типичные причины уязвимостей

На вводном уровне особенно важно понимать несколько опасных функций и паттернов:

- `gets()` — не проверяет длину ввода и сразу всё записывает
- `scanf("\%s")` — не ограничивает размер считываемой строки
- `strcpy()` — копирует строку без проверки размера буфера назначения
- `strcat()` — конкатенирует строки без проверки размера
- `printf(buf)` — небезопасный форматный вывод

Эти функции становятся проблемой, когда программа не контролирует длину данных, формат или корректность адресов.

Основные классы ошибок:

- Переписывание строки - если на строку выделено 4 байта, а ввести больше, то строка радом перезапишется
- Переписывание указателя - аналогично
- Переполнение целого числа

$imp

Главная идея бинарных уязвимостей состоит в том, что ошибка в данных может превратиться в ошибку управления. Это и делает такие уязвимости особенно опасными.

$end

Архитектура x86_64, стек и переполнение стека

Третье занятие посвящено низкоуровневой основе анализа бинарных уязвимостей. Чтобы понимать переполнение стека, недостаточно знать только общую теорию: необходимо представлять, как устроены регистры, как передаются аргументы в функции, как формируется стековый кадр и что именно хранится в памяти функции.

Основы архитектуры x86_64

В модуле рассматривается архитектура x86_64 как наиболее распространённая для настольных и серверных систем. Она использует 64-битные регистры и предоставляет набор инструкций, необходимых для работы с арифметикой, памятью, переходами и вызовами функций.

Основные регистры общего назначения:

- RAX — регистр, часто используемый для арифметики и возвращаемого значения
- RBX, RCX, RDX — регистры общего назначения
- RSP — указатель на вершину стека
- RBP — базовый указатель текущего стекового кадра
- RDI, RSI — регистры для передачи аргументов
- RIP — указатель на текущую инструкцию
- RFLAGS — регистр флагов состояния

$term

RSP — это регистр, указывающий на вершину стека.

$end

$term

RBP — это регистр, фиксирующий базу текущего стекового кадра функции.

$end

$term

RIP — это регистр, указывающий на адрес инструкции, которая будет выполняться процессором.

$end

Размеры данных:

- Byte — 1 байт
- Word — 2 байта
- Dword — 4 байта
- Qword — 8 байт

Флаги в RFLAGS:

- `CF` — флаг переноса
- `ZF` — флаг нуля
- `OF` — флаг переполнения
- `DF` — флаг направления

Основные инструкции ассемблера

На базовом уровне для понимания ассемблерного кода достаточно нескольких инструкций:

- `mov` — перемещение данных
- `add` — сложение
- `sub` — вычитание
- `cmp` — сравнение
- `jmp` — безусловный переход
- `je`, `jne`, `jg`, `jl` — условные переходы
- `nop` — нет операции (no operation)

После арифметических операций и сравнений изменяются флаги состояния в регистре `RFLAGS`. Эти флаги затем используются для принятия решений в условных переходах.

Устройство стека

$term

Стек — это область памяти, в которой хранятся локальные переменные функции, временные данные, сохранённые регистры и адреса возврата.

$end

Стек организован по принципу LIFO: последним пришёл — первым вышел. При вызове функций и работе с локальными переменными стек постоянно изменяется. В архитектуре x86_64 стек растёт в сторону уменьшения адресов.

Инструкции работы со стеком:

- `push rax` — уменьшает `RSP` и кладёт значение на стек
- `pop rax` — считывает значение со стека и увеличивает `RSP`
- `call func` — сохраняет адрес возврата и передаёт управление функции
- `ret` — извлекает адрес возврата и возвращает управление

Стековый кадр функции

Каждая функция обычно создаёт собственный стековый фрейм, чтобы хранить локальные данные и работать с аргументами в устойчивой структуре.

$case

Типичный пролог функции:

```
push rbp
mov rbp, rsp
sub rsp, 32
```

Типичный эпилог функции:

```
mov rsp, rbp
pop rbp
ret
```

Такая схема позволяет функции сохранить старое состояние стека, выделить место под локальные переменные, а затем корректно восстановить контекст перед возвратом.

$end

Calling convention

Для анализа функций важно понимать соглашение о вызовах функций.

В x86_64 первые шесть целочисленных аргументов обычно передаются через регистры RDI, RSI, RDX, RCX, R8, R9. Если аргументов больше, остальные передаются через стек. Возвращаемое значение обычно помещается в RAX.

Это соглашение позволяет при чтении дизассемблированного кода понимать, откуда функция получает входные данные и куда помещает результат.

Также важно различать caller-saved и callee-saved регистры, что влияет на то, какие регистры сохраняются вызывающей функцией, а какие — вызываемой.

Переполнение стека

$term

Переполнение стека — это запись данных за пределы выделенного локального буфера в стековой памяти.

$end

Такое переполнение особенно опасно, потому что рядом с локальными буферами в стековом кадре находятся служебные значения — сохранённый RBP и адрес возврата. Если программа записывает в буфер слишком много данных и не контролирует длину ввода, злоумышленник может повредить эти значения.

В простейшем случае переполнение приводит к аварийному завершению программы. В более опасном варианте атакующий получает возможность изменить адрес возврата и перенаправить управление на нужный участок кода.

$imp

Переполнение стека опасно не само по себе, а потому, что может затронуть управляющие данные функции. Именно это превращает ошибку записи в потенциальный захват управления выполнением программы.

$end

Защиты от эксплуатации

Современные системы используют несколько механизмов, которые затрудняют эксплуатацию бинарных уязвимостей.

$term

ASLR (Address Space Layout Randomization) — это рандомизация адресов загрузки библиотек и других участков памяти процесса.

$end

$term

PIE (Position Independent Executable) — это механизм, позволяющий рандомизировать базовый адрес самого исполняемого файла.

$end

$term

Stack Canary — это специальное контрольное значение, размещаемое в стековом кадре и проверяемое перед возвратом из функции.

$end

$term

Full RELRO (Relocation Read-Only) — это режим защиты, при котором таблица GOT становится недоступной для записи после инициализации.

$end

Эти механизмы не устраняют причину уязвимости, но значительно усложняют практическую эксплуатацию ошибки. Поэтому при анализе бинарных программ важно уметь оценивать не только наличие уязвимости, но и активные механизмы защиты.

Итоги модуля

В рамках модуля мы сформировали целостное представление о реверс-инжиниринге и бинарных уязвимостях. Сначала были рассмотрены цели и области применения реверса, затем — различия между статическим и динамическим анализом, после чего внимание было сосредоточено на структуре исполняемых файлов, инструментах отладки, устройстве памяти и особенностях архитектуры x86_64.

Отдельным результатом модуля стало понимание того, как низкоуровневые детали реализации функций связаны с безопасностью программы. Вы должны видеть связь между буфером на стеке, адресом возврата, инструкциями call и ret, а также понимать, почему переполнение стека является одним из базовых и самых важных классов бинарных уязвимостей.

Полученные знания образуют фундамент для дальнейшего изучения эксплуатации уязвимостей, написания эксплойтов, анализа вредоносного ПО и продвинутой работы с отладчиками и дизассемблерами.

Полезные материалы для дальнейшего изучения

$biblio

- Tasks :: forkbomb.ru - видео и задачи для тренировки
- Flare-On - шикарный CTF для ревёрсеров. Сложность постепенно растёт от первой до последней задачи

- play.picoctf.org - хороший сайт с задачами
- Введение в реверсинг с нуля, используя IDA PRO - курс по реверсу
- К. Касперски “Искусство дизассемблирования” - старая, но хорошая книга.
- Сикорски М., Хониг Э. “Вскрытие покажет! Практический анализ вредоносного ПО” - хорошая книга про анализ ВПО
- Chris Eagle “The IDA Pro Book: The Unofficial Guide to the World’s Most Popular Disassembler”
- Шпаргалка полезных команд GDB / Хабр
- GDB Advanced Debugging Skills | Axura
- https://pwndbg.re/stable/commands/
- pwn.spbctf.ru/tasks
- pwn.college
- Yet Another Pwntools Cheatsheet | 🐕 . 🪦
- Руководство по ассемблеру NASM

$end